

0.66. La lettre bouge avec la croix

Le joueur agit, l'écran répond : c'est déjà un jeu.

1. Ce qu'il faut comprendre

`bouton(DROITE)` vaut vrai **tant que** la flèche droite est enfoncée. Même chose pour `GAUCHE`, `HAUT` et `BAS`.

Bouger à chaque image ferait traverser l'écran en un tiers de seconde. `attente` impose donc **8 images** entre deux pas : en gardant la flèche enfoncée, la lettre avance d'une case environ sept fois par seconde.

Les conditions `x < 19`, `x > 0`, `y < 17` et `y > 0` empêchent de sortir de l'écran.

Voilà les trois ingrédients d'un jeu : la **boucle**, le **joueur qui agit**, et l'**écran qui répond**.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1602 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 9;           // la lettre part du milieu de l'écran
```

2. UNE VARIABLE — Y

```
uint8_t y = 8;
```

3. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0; // les images à attendre avant le prochain pas
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        if (attente > 0) attente--;

        if (attente == 0) {
            effacer(x, y, 1);           // efface l'ancienne place
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

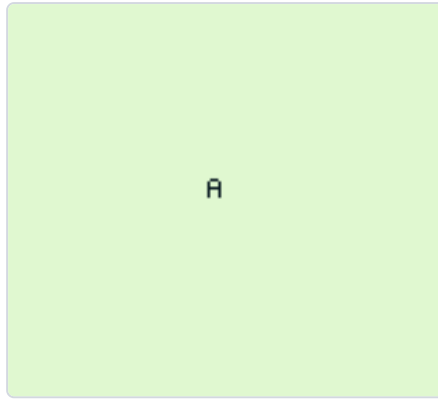
        poser(x, y, ALPHABET[0]);       // la lettre à sa place
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



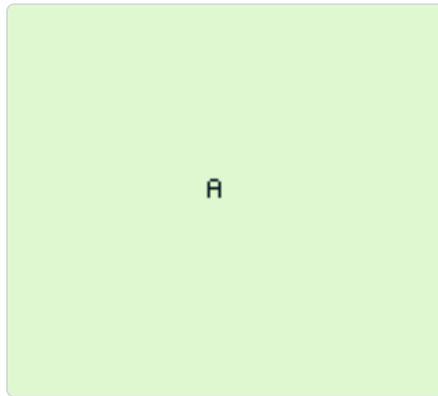
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

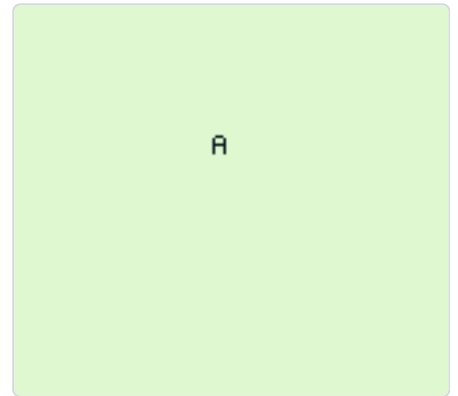
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



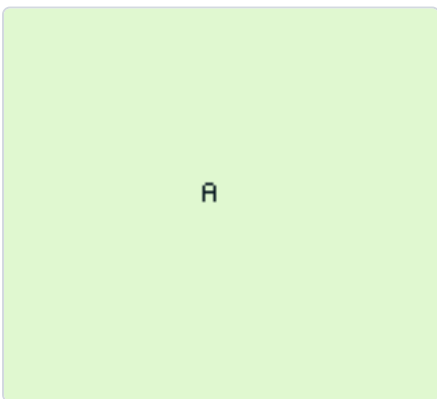
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un A au milieu de l'écran, qui se déplace avec les flèches.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A part du milieu

✓ la flèche droite le déplace

✓ la flèche bas aussi

✓ il est à sa nouvelle place

✓ et plus au départ

Ensuite : **0.66.1. La lettre bouge avec la croix — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++